Comprehensive Architecture and Compliance Report for a Cross-Platform Public Transit Telemetry System

The modernization of public transit tracking requires robust, distributed systems capable of ingesting, processing, and disseminating real-time telemetry data. For a commuter network as complex as the S-Bahn München (Munich S-Bahn), which is deeply integrated into the Münchner Verkehrs- und Tarifverbund (MVV) and operated by Deutsche Bahn (DB), capturing high-fidelity delay data poses significant architectural, economic, and legal challenges. The requirement to monitor this data at least every five minutes and subsequently generate advanced statistical metrics necessitates a highly fault-tolerant ingestion engine. Furthermore, the mandate that this application must operate seamlessly across a local machine, within the Google Cloud Platform (GCP), and natively on an Android device dictates a strictly cross-platform architectural approach.

This report presents an exhaustive architectural blueprint for a highly secure, privacy-compliant, and cost-optimized telemetry application. Golang (Go) serves as the primary programming language, leveraged for its exceptional concurrency models, cross-platform compilation capabilities, and low resource overhead. By designing the core engine in Go, the system achieves maximum code reuse, allowing the exact same business logic to run as a local system daemon, a serverless cloud container, and a compiled native library on Android.

Legal and Regulatory Compliance Framework

The acquisition, storage, and processing of public transit telemetry data intersect heavily with European regulatory frameworks. Any system interacting with external application programming interfaces (APIs) or web endpoints must be rigorously evaluated against the General Data Protection Regulation (GDPR), the German Federal Data Protection Act (BDSG), the Telecommunications Digital Services Data Protection Act (TDDDG), and the emerging provisions of the European Data Act.

The Legal Risks of Web Scraping and Database Rights

Historically, developers seeking S-Bahn München delay data resorted to web scraping techniques, extracting punctuality information from HTML interfaces such as the Hafas system or the MVV backend.[1] However, scraping is structurally brittle, highly susceptible to interface modifications, and legally contentious under evolving European data regulations.[2]

Personal data does not lose its protections under GDPR simply by being published online, and the collection of publicly available data must still comply with GDPR principles.[2] The European

Data Protection Supervisor has expressed severe concerns regarding web scraping, particularly emphasizing the loss of control individuals experience over their data when it is processed beyond their expectations.[2] More critically for this architecture, recent jurisprudence from the German Federal Court of Justice (BGH) in November 2024 (Docket No.: VI ZR 10/24) dealt with claims for non-material damages pursuant to Article 82 GDPR following a scraping incident.[4] The BGH ruled that a proven loss of control or a well-founded fear of misuse of scraped data by third parties is sufficient to establish non-material damage, paving the way for substantial compensation claims.[4]

Beyond personal data, corporate databases are protected by intellectual property laws and Database Directives. Transit agencies actively defend their infrastructure from unauthorized automated access. Therefore, extracting data via unauthorized web scraping carries an unacceptable legal and financial risk profile for an enterprise-grade application. The architectural shift to using official Open Data interfaces is not merely a technical optimization; it is a strict legal necessity.

The European Data Act (2025/2026) and Open Data Mandates

The legal landscape for data acquisition has fundamentally shifted in favor of open access due to the European Data Act, which entered into application on September 12, 2025.[6] The Data Act is a comprehensive initiative designed to empower users and businesses by providing fair access to data generated by connected devices and infrastructure.[7] It prohibits unfair contracts that prevent data sharing and mandates that entities operating essential infrastructure make non-personal data available under fair, reasonable, and non-discriminatory (FRAND) terms.[7]

For a public transit telemetry system, the Data Act operates as a powerful regulatory tailwind. It reinforces the mandate established by the EU Commission's Delegated Regulation 2017/1926, which requires the publication of national static timetable data via a national access point.[10] This ensures that the availability of official transit APIs—such as those aggregating S-Bahn München data—will remain stable and legally protected. By utilizing these officially sanctioned open data streams, the application operates entirely within the bounds of European intellectual property and database laws.

GDPR Privacy by Design and Data Minimization

While the application primarily ingests non-personal train delay telemetry, GDPR compliance risk arises through the secondary data collection of the application's users, particularly in the Android deployment. The GDPR defines personal data broadly, encompassing location data, IP addresses, and unique device identifiers.[3] If the Android application tracks a user's location to filter nearby delayed S-Bahn trains, this constitutes high-risk personal data processing.

To maintain the highest privacy standards and minimize compliance overhead, the architecture must implement "Privacy by Design" and "Privacy by Default" as mandated by GDPR Article 25.[11] This requires that the most privacy-friendly settings are applied by default, such as requiring explicit opt-in consent for location tracking and avoiding the collection of any data that is not strictly necessary for the application's core functionality.[11]

Crucially, the architecture relies on local processing exemptions to mitigate regulatory exposure. According to application marketplace standards and GDPR data minimization principles, user data that is accessed by the application but processed strictly locally on the user's device—and never transmitted off-device to a backend server—falls outside the scope of reportable data collection.[15] By executing the telemetry engine locally on the Android device, downloading the transit data to the phone, and performing any location-based filtering entirely on the client side without transmitting the user's coordinates, the application vastly reduces its GDPR compliance burden.

Furthermore, the employment of end-to-end encryption for any data transmitted off-device, and robust local encryption for data at rest, fulfills the GDPR Article 32 requirements for implementing appropriate technical and organizational measures to ensure data security.[13]

Data Acquisition Strategy and Protocol Specifications

The foundational layer of the telemetry system is its data acquisition mechanism. The application must poll S-Bahn München delay data at least every five minutes. To achieve this reliably, the system must interact with highly available, standardized data feeds rather than relying on disparate, undocumented backend APIs.

Primary Data Source: GTFS-Realtime (GTFS-RT)

The optimal method for continuous transit data ingestion is the General Transit Feed Specification Realtime (GTFS-RT) protocol. GTFS-RT is an industry-standard specification that allows transit agencies to provide consumers with real-time information about disruptions to their service, including station closures, line cancellations, and, most importantly, expected arrival times and delays (via TripUpdates).[18]

For the German transit network, highly reliable, aggregated GTFS-RT streams are provided by the open-data platform gtfs.de.[19] This platform synthesizes real-time data published by transportation companies under open licenses or special agreements to match their static GTFS feeds.[19] The static feeds cover over 20,000 lines, more than 500,000 stop points, and nearly 2 million regular vehicle journeys, representing one of the largest GTFS datasets globally.[10]

For the Bavarian region, which encompasses the S-Bahn München, transit data is fundamentally sourced from DEFAS Bayern (Digitales Daten- und Hintergrundsystem für den ÖPNV in Bayern).[19] DEFAS serves as the central dynamic background system for public

mobility in the state, integrating real-time data from over 100 transport companies and associations, including the Münchner Verkehrs- und Tarifverbund (MVV) and Deutsche Bahn.[19] The gtfs.de real-time stream aggregates this data and is updated every 10 seconds, which is vastly superior to the 5-minute polling interval requirement of this application.[19]

The GTFS-RT data is serialized using Protocol Buffers (protobuf), a highly efficient, language-neutral binary format developed by Google. Protobuf minimizes payload size and accelerates parsing speeds compared to verbose formats like XML or JSON. The data is provided under a Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0) license.[19] The application must comply with this license by explicitly attributing the data source within the user interface and ensuring any derivative distributed datasets remain under the same license terms.[19]

Contingency Planning and Fallback Architectures

System resilience dictates the necessity of a fallback mechanism should the primary GTFS-RT stream experience degradation or if S-Bahn München data specifically becomes unavailable. The application architecture must be capable of dynamically pivoting to alternative data sources without requiring a complete rewrite of the core engine.

Contingency Scenario	Primary Strategy	Technical Implementation	Data Format
gtfs.de API Outage	Failover to Official DB API	Implement an adapter for the Deutsche Bahn Fahrplan API, which provides schedules and real-time information for trains and public transport across Germany.[20]	XML / JSON [20]
S-Bahn München Data Obfuscation	Pivot to Alternative Regions	Since the system parses standard GTFS-RT, modifying the route_id and agency_id configuration	Protobuf [18]

		parameters allows the system to instantly monitor alternative transit networks (e.g., Berlin VBB, Hamburg HVV) present in the nationwide GTFS feed.[10]	
Total Real-Time Blackout	Statistical Inference	Utilize historical delay datasets to compute the statistical probability of a delay based on time, day, and historical precedent.	Parquet / SQL [21]

If real-time data becomes entirely inaccessible, the application gracefully degrades to statistical inference. Historical delay data can be bootstrapped into the system using open-source repositories such as deutsche-bahn-data, which aggregates monthly processed data of train delays across Germany into Parquet files.[21] By applying predictive algorithms to this historical baseline, the system can estimate the likelihood of delays, displaying a confidence interval to the user rather than a live metric.[22] This approach mirrors analytical projects like zugspaet.de, which calculate how often specific trains were late in the past.[23]

Core Telemetry Engine Architecture (Golang)

The core application logic is constructed entirely in Golang. Go's compilation model allows the exact same business logic to be compiled into a statically linked Linux executable for local machines, a lightweight container for Google Cloud, and an Android Archive (AAR) for mobile devices.[24] Go is chosen for its exceptional concurrency features (goroutines), its minimal memory footprint, and its avoidance of heavy virtual machines (like the JVM), making it ideal for continuous background polling.[24]

Interface-Driven Design Pattern

To accommodate the multi-platform requirement (Local, GCP, Android) and the multi-source contingency plan (GTFS-RT, DB API), the system utilizes a strictly interface-driven architectural pattern, often referred to as Ports and Adapters or Hexagonal Architecture. The core polling engine does not interact directly with the operating system, the network protocols, or specific

database drivers. Instead, it relies on abstract interfaces.

The TransitProvider interface defines the contract for fetching data. It requires methods such as FetchDelays(routeID string) (DelayRecord, error). The primary implementation is the GTFSRTProvider, which handles the network requests to gtfs.de and the protobuf unmarshaling. If the contingency plan is activated, the engine seamlessly swaps to the DBApiProvider implementation.

Similarly, the StorageProvider interface defines the contract for persisting data and retrieving metrics, utilizing methods like SaveRecords(recordsDelayRecord) and GetAggregatedMetrics(routeID string) (Metrics, error). The implementations of this interface vary drastically based on the deployment target: SQLiteStorage for Local and Android environments, and BigQueryStorage for the Google Cloud deployment.

GTFS-RT Parsing and Concurrency Optimization

The five-minute polling constraint is managed via Go's native time.Ticker in standalone environments (Local and Android). When the ticker fires, the engine utilizes goroutines to execute non-blocking HTTP GET requests to the GTFS-RT endpoints.

To handle the Protocol Buffer payload, the architecture utilizes the gtfs-realtime-bindings library provided by MobilityData, or a community-maintained parser such as cedarbaum/gtfs.[26] These libraries provide pre-generated Golang structs from the GTFS-realtime specification, allowing the application to parse the binary feed directly into Go objects.[27]

Given that the aggregated GTFS-RT feed for the entirety of Germany can be substantial in size, memory management is a critical concern, particularly for the Android deployment. The Go implementation employs io.Reader streams rather than loading the entire binary payload into RAM simultaneously. The parser iterates through the FeedMessage entities, extracting the TripUpdate objects.[27] The custom parsing logic then cross-references the trip_id and route_id to filter out everything except the target S-Bahn München lines.[28] Furthermore, the parser converts the raw UNIX timestamps embedded in the protobuf into idiomatic time.Time values, facilitating easier temporal analysis downstream.[26]

Cloud Infrastructure and Economic Optimization (GCP)

Deploying a continuous polling system in the cloud introduces the risk of runaway operational costs. However, by leveraging serverless architectures within Google Cloud Platform (GCP), the total cost of ownership can be driven close to zero while maintaining enterprise-grade reliability and scalability.

Compute Layer: Cloud Run Jobs vs. Cloud Functions

To execute the Golang engine every five minutes, the architecture mandates the use of GCP Cloud Run Jobs coupled with Cloud Scheduler. This architectural decision explicitly rejects the use of long-running virtual machines (Compute Engine) or first-generation Cloud Functions.[30]

While Cloud Functions and Cloud Run Services are designed to respond to incoming HTTP requests, Cloud Run Jobs are engineered specifically for unattended, scheduled batch execution without the need to expose a web server or maintain a listening framework.[31] A Cloud Scheduler instance is configured with a standard cron expression (*/5 * * * *) to trigger the Cloud Run Job exactly every five minutes.[32]

Economically, Cloud Run is billed on a pay-as-you-go model, charging only for the resources actually used, rounded up to the nearest 100 milliseconds.[30] The billing encompasses CPU and memory allocation, plus a minor fee per request beyond the free quota.[30]

A five-minute polling interval results in exactly 288 executions per day. Assuming the highly optimized Go binary requires 1.5 seconds to fetch the GTFS-RT stream, parse the protobuf, and stream the relevant records to the database, the total daily compute time is approximately 432 seconds. Over a 30-day month, this totals roughly 12,960 seconds of execution time. This falls vastly below the generous Cloud Run free tier, which provides 180,000 vCPU-seconds and 360,000 GiB-seconds per month.[34] Furthermore, Cloud Scheduler provides three free jobs per month per billing account, ensuring the trigger mechanism itself incurs zero cost.[32] By carefully right-sizing the container (e.g., allocating 0.5 vCPU and 256MB RAM), the compute costs for the cloud deployment will remain entirely within the free tier.[30]

Storage and Analytics Layer: BigQuery

For storing the continuous stream of time-series delay data and performing complex statistical aggregations, Google BigQuery is the overwhelmingly superior choice over relational databases like Cloud SQL or NoSQL document stores like Firestore.[35]

Database Service	Architectural Paradigm	Cost Profile for Time-Series	Limitations for this Specific Use Case
Cloud SQL	Relational / ACID	High (Fixed hourly cost)	Massive overkill for append-only telemetry data. Requires provisioning

			instances, resulting in a high baseline cost regardless of usage.[37]
Firestore	NoSQL / Document	Moderate (Per-read/write billing)	The free tier allows 20,000 writes per day. Writing multiple train delay updates 288 times a day could quickly exceed the free tier limits, leading to escalating costs.[39]
BigQuery	Serverless Columnar Analytics	Extremely Low (Pay per byte queried)	Ideal for this architecture. BigQuery scales seamlessly and is incredibly cost-effective for append-only storage and massive aggregations.[35]

BigQuery natively supports time-series analysis and provides specialized time-bucketing functions (TIMESTAMP_BUCKET, DATE_BUCKET, and DATETIME_BUCKET) that make calculating hourly, daily, or weekly average delays mathematically trivial.[40] These functions map input time values to specific windows, allowing the aggregation of multiple data points using functions like AVG, MIN, MAX, or COUNT.[40]

The Golang telemetry engine utilizes the BigQuery Storage Write API to stream the delay metrics directly into a highly optimized table. To ensure performance and cost-efficiency, the table is strictly partitioned by the ingestion_timestamp and clustered by the route_id (e.g., indicating lines S1, S2, S3, etc.). This schema design ensures that when the application queries the database to generate metrics, BigQuery only scans the specific partitions and clusters required, keeping query sizes minimal. Given that BigQuery offers a massive free tier of 1 TB of querying per month and 10 GB of storage, the storage and analytical costs for this telemetry application will be effectively zero.[36]

Local Machine Deployment Architecture

For developers, system administrators, or data analysts wishing to run the application continuously on a local Linux environment (e.g., a Raspberry Pi or a dedicated home server), the Go application cross-compiles to a standalone, statically linked binary. This eliminates the need for complex runtime environments or dependency management.

Systemd Integration for Background Execution

To ensure the application runs continuously in the background, survives system reboots, and recovers from unexpected crashes, it must be registered as a systemd daemon.[42] The architecture specifies the creation of a standard systemd unit file (e.g., /etc/systemd/system/sbahn-telemetry.service).

Best practices dictate that the service must not run as the root user to mitigate potential security vulnerabilities; a dedicated unprivileged user account should be utilized.[42] The systemd configuration utilizes Type=simple, which is standard for modern background processes that do not rely on legacy forking behavior.[43] Standard output and standard error streams generated by the Go application are automatically captured by journald. This provides comprehensive, rotatable logging and debugging capabilities (journalctl -u sbahn-telemetry.service) without requiring a dedicated, complex logging framework within the Go application itself.[43]

Furthermore, resource limits are strictly enforced using systemd cgroups. By defining parameters such as MemoryLimit=256M within the service file, the operating system guarantees that the local machine is not starved of RAM in the highly unlikely event of a memory leak within the Go application.[46] Once the service file is created, the system administrator executes sudo systemctl daemon-reload followed by sudo systemctl enable --now sbahn-telemetry.service to initialize the continuous polling engine.[42]

Local Storage Implementation: SQLite with WAL

When running locally, connecting to a cloud database like BigQuery introduces unnecessary latency, network dependencies, and potential costs. Therefore, the StorageProvider interface injects a local SQLite database utilizing the mattn/go-sqlite3 driver.

To support concurrent operations—where the background ticker continuously writes new delay data every five minutes while a local web dashboard or CLI tool simultaneously queries the database for statistics—SQLite must be configured appropriately. The architecture mandates enabling Write-Ahead Logging (WAL) mode by executing the pragma statement PRAGMA journal_mode=WAL; upon initialization. WAL mode significantly improves concurrency by allowing readers to operate simultaneously with a single writer, eliminating the dreaded "database is locked" errors common in highly active SQLite deployments.

Android Native Deployment Architecture

Deploying a continuous 5-minute polling engine on the Android operating system introduces severe architectural constraints. Modern Android versions aggressively terminate background processes to preserve battery life, manage memory, and prevent malicious tracking (e.g., Doze mode and App Standby Buckets).[47] Designing an architecture that reliably executes network requests every five minutes on a mobile device requires careful navigation of the Android lifecycle.

UI and Native Integration: Gomobile vs. Fyne

There are two primary paradigms for executing Golang code on Android devices: utilizing a pure Go UI toolkit or employing a hybrid binding approach.

Approach	Technology	Pros	Cons	Architectural Verdict
Pure Go UI	Fyne (fyne.io)	Single language for UI and backend; cross-platform UI code.[24]	Android OS suspends Fyne applications almost immediately after they lose foreground visibility.[47] No robust path to extend Android Service classes entirely in Go.[47]	**Rejected.** The inability to maintain continuous background execution violates the core 5-minute polling requirement.[47]
Hybrid Native	Gomobile Bind	Generates native bindings (AAR); allows UI to be written in Kotlin; excellent performance.[24]	Requires maintaining a Kotlin codebase for the UI and Android lifecycle management.[50]	**Selected.** Allows the core Go engine to be wrapped within an Android foreground service, guaranteeing execution.[52]

The hybrid approach using the official gomobile bind tool is strictly mandated for this architecture. The tool compiles the core Go telemetry engine into a native Android Archive (.aar)

library.[24] This exposes the Go functions to the Android environment via the Java Native Interface (JNI).[54] The user interface, system permissions, and operating system lifecycle management are handled natively in Kotlin, while the high-performance GTFS parsing, data structuring, and statistical computations are entirely encapsulated within the Go library.[50]

To optimize performance and minimize the overhead of crossing the JNI boundary, the architecture dictates coarse-grained calls.[25] Instead of the Kotlin layer fetching the massive GTFS-RT binary payload and passing it to Go for parsing (which would cause severe memory churn), the Go layer handles both the HTTP request and the parsing internally. It only returns primitive types, simple structs, or small JSON strings back to the Kotlin UI layer.[25]

Overcoming Android Background Execution Limits

To guarantee the execution of the telemetry poll every five minutes, standard Android background processing tools are insufficient. For instance, the Android WorkManager API, which is generally recommended for deferrable tasks, enforces a strict minimum periodic interval of 15 minutes. To adhere strictly to the 5-minute requirement, the application must utilize an Android **Foreground Service**.[55]

A Bound Foreground Service signals to the Android operating system that the application is performing a task the user is actively aware of.[55] It requires a persistent, non-dismissible notification to be displayed in the system tray while the service is active.[55] Within this service, a Kotlin coroutine is established. The coroutine utilizes a timer to invoke the Go telemetry function via the gomobile bindings exactly every five minutes.[52] To prevent the device from entering deep sleep during the brief polling window, the service may need to acquire a temporary WakeLock via the Android PowerManager, though this must be managed extremely carefully to avoid draining the user's battery.[55] If the data cannot be fetched due to transient network loss, the Go logic caches the state and retries dynamically on the next cycle.

Encrypted Local Storage and Privacy Security

Mobile devices are highly susceptible to loss, theft, or unauthorized access. Therefore, any telemetry data or user preferences stored on the device must be heavily secured to meet the "highest privacy standards" required by the system specification.

The Android application architecture dictates the use of SQLCipher, an open-source extension to SQLite that provides transparent, on-the-fly 256-bit AES encryption of database files.[16] Full disk encryption, while standard on modern Android devices, only protects data when the device is powered off; SQLCipher protects the application's specific database even while the device is running.[17]

The encryption key used by SQLCipher is generated securely and stored within the hardware-backed Android Keystore system. This architecture ensures that even if the device is

compromised or subjected to forensic analysis, the telemetry database remains entirely unreadable to unauthorized actors.[16] This robust local encryption satisfies the stringent security requirements of GDPR Article 32, reducing the probability of a personal data breach.[16]

Statistical Modeling, Metrics, and Analytics

The raw telemetry data gathered from the GTFS-RT feed is highly voluminous, consisting of constant streams of vehicle coordinates, trip updates, and timestamp deltas. Storing this data provides limited immediate value without the application of statistical aggregation to generate actionable metrics. The core Go engine, depending on its deployment environment, applies multiple algorithms to distill this raw data into meaningful insights.

Core Metrics Generation

The system is designed to calculate several key performance indicators regarding the S-Bahn München network:

1. **Punctuality Ratio:** This is the foundational metric, defined as the percentage of trains arriving within specific threshold brackets. Deutsche Bahn traditionally defines a train as "on-time" if it arrives within 5 minutes and 59 seconds of its scheduled arrival. The system calculates the ratio of trips falling under this threshold versus those exceeding it.
2. **Percentile Delays:** Average (mean) delays are often heavily skewed by catastrophic network failures or extreme outliers. Therefore, the architecture computes the 50th (median), 90th, and 95th percentiles of delay times. This provides a much more accurate representation of the typical commuter experience.
3. **Temporal and Spatial Aggregation:** The system analyzes the data to identify specific operational windows (e.g., morning and evening rush hours) and specific stations that statistically incur the highest propagation of delays.[22] This mirrors academic research methodologies that track how transit delays cascade through a network and influence multi-modal transit demand.[57]

Deployment-Specific Analytical Engines

The mechanism for computing these statistics varies fundamentally based on the deployment target:

In the Google Cloud Platform deployment, the heavy lifting of statistical calculation is entirely offloaded to BigQuery's massively parallel SQL engine. The Go application simply streams the raw DelayRecord structs into the database. Metrics are generated using advanced analytical SQL capabilities. For example, rolling averages and percentiles are computed using BigQuery window functions (e.g., PERCENTILE_CONT(delay_seconds, 0.90) OVER(PARTITION BY route_id) or AVG(delay_seconds) OVER (PARTITION BY route_id ORDER BY timestamp ROWS BETWEEN 12 PRECEDING AND CURRENT ROW)). This approach allows the cloud application to serve highly complex dashboards instantly without consuming compute resources

in the Cloud Run container.

Conversely, in the local and Android deployments, executing complex analytical queries against a local SQLite database can be CPU-intensive and drain battery life. Therefore, the Go engine itself is responsible for calculating these metrics. The StorageProvider interface retrieves the raw records from SQLite, and the Go application utilizes highly optimized, in-memory statistical algorithms to compute the punctuality ratios and percentiles before presenting the final metrics to the user interface. This ensures the SQLite database operates primarily as an append-only ledger, maximizing write performance during the 5-minute polling cycles.

Conclusion

The design of a cross-platform public transit telemetry application for the S-Bahn München network demands a sophisticated orchestration of systems engineering, cloud economics, and strict adherence to European data regulations. By anchoring the core architecture in Golang, the system achieves unprecedented code reuse, allowing a single, highly concurrent engine to operate across drastically different environments: running quietly as a systemd daemon on a local machine, executing as a hyper-efficient, cost-free Cloud Run Job within GCP, and running natively as an integrated background service on Android via gomobile bind.

Crucially, the architectural pivot away from brittle web scraping techniques toward the integration of official GTFS-RT streams circumvents significant legal liabilities. It ensures compliance with the strict precedents set by recent judicial rulings on data scraping and aligns perfectly with the open data mandates of the incoming EU Data Act. By utilizing BigQuery for serverless cloud analytics and mandating AES-256 SQLCipher encryption on Android devices, the application guarantees both near-zero operational costs and absolute adherence to GDPR Privacy by Design principles. This comprehensive architecture provides an exhaustive, scalable, and enterprise-grade blueprint, ready for immediate engineering implementation.

Referenzen

1. MVVLive - PyPI, Zugriff am März 21, 2026, https://pypi.org/project/MVVLive/
2. To scrape or not to scrape: EU authorities' recent interpretations - Dentons, Zugriff am März 21, 2026, https://www.dentons.com/en/insights/articles/2024/june/18/to-scrape-or-not-to-scrape-eu-authorities
3. The state of web scraping in the EU - IAPP, Zugriff am März 21, 2026, https://iapp.org/news/a/the-state-of-web-scraping-in-the-eu
4. EU/Germany: Damages after data breach/scraping – Groundbreaking case law, Zugriff am März 21, 2026, https://www.reedsmith.com/articles/eu-germany-damages-after-data-breach-scraping-groundbreaking-case-law/
5. Compensation for data leaks under Art. 82 GDPR? The current legal situation regarding data scraping | Maucher Jenkins, Zugriff am März 21, 2026,

https://www.maucherjenkins.com/en/news-and-events/commentary/compensation-for-data-leaks-under-art-82-gdpr-the-current-legal-situation-regarding-data-scraping/
6. EU & German Data Protection: GDPR, AI Act, Data Act Updates - CMS, Zugriff am März 21, 2026, https://cms.law/en/deu/legal-updates/2025-in-data-protection
7. Data Act | Shaping Europe's digital future - European Union, Zugriff am März 21, 2026, https://digital-strategy.ec.europa.eu/en/policies/data-act
8. EU Data Act and contractual changes – what to do? - BDO, Zugriff am März 21, 2026, https://www.bdolegal.de/en-gb/insights-en/updates/2025/eu-data-act-and-contractual-changes-what-to-do
9. Data law | ITM, Zugriff am März 21, 2026, https://www.itm.nrw/wp-content/uploads/2025/06/Script-Data-Law-February-2025-Final.pdf
10. GTFS for Germany, Zugriff am März 21, 2026, https://gtfs.de/en/
11. Data protection under GDPR - Your Europe - European Union, Zugriff am März 21, 2026, https://europa.eu/youreurope/business/dealing-with-customers/data-protection/data-protection-gdpr/index_en.htm
12. GDPR - Application privacy by design - IBM Developer, Zugriff am März 21, 2026, https://developer.ibm.com/articles/s-gdpr2/
13. Privacy by Design & Default (GDPR): Implementation Guide, Zugriff am März 21, 2026, https://secureprivacy.ai/blog/privacy-by-design-gdpr-2025
14. GDPR Compliance for Apps - Privacy Policies, Zugriff am März 21, 2026, https://www.privacypolicies.com/blog/gdpr-compliance-apps/
15. Provide information for Google Play's Data safety section - Play Console Help, Zugriff am März 21, 2026, https://support.google.com/googleplay/android-developer/answer/10787469?hl=en
16. Encryption - General Data Protection Regulation (GDPR), Zugriff am März 21, 2026, https://gdpr-info.eu/issues/encryption/
17. Encryption and data storage | ICO - Information Commissioner's Office, Zugriff am März 21, 2026, https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/security/encryption/encryption-and-data-storage/
18. GTFS Realtime Reference, Zugriff am März 21, 2026, https://gtfs.org/documentation/realtime/reference/
19. Realtime data - GTFS for Germany, Zugriff am März 21, 2026, https://gtfs.de/en/realtime/
20. DB Fahrplan API API details - facts.dev, Zugriff am März 21, 2026, https://www.facts.dev/api/db-fahrplan-api/
21. piebro/deutsche-bahn-data - GitHub, Zugriff am März 21, 2026, https://github.com/piebro/deutsche-bahn-data
22. Analysis of delays of the Deutsche Bahn (DB) - GitHub, Zugriff am März 21, 2026, https://github.com/Splines/deutsche-bahn-analysis
23. GitHub - AlexW00/zugspaet: Check if your DB train was late in the past, Zugriff am

März 21, 2026, https://github.com/AlexW00/zugspaet
24. Golang for Mobile App Development: Benefits, Tools & Future - 21Twelve Interactive, Zugriff am März 21, 2026, https://www.21twelveinteractive.com/why-choose-golang-for-mobile-app-development/
25. I made go run on mobile (Android / iOS) -> React Native JSI + GoMobile setup - Reddit, Zugriff am März 21, 2026, https://www.reddit.com/r/golang/comments/1nuad5s/i_made_go_run_on_mobile_android_ios_react_native/
26. gtfs package - github.com/cedarbaum/gtfs - Go Packages, Zugriff am März 21, 2026, https://pkg.go.dev/github.com/cedarbaum/gtfs
27. Golang GTFS-realtime Language Bindings, Zugriff am März 21, 2026, https://gtfs.org/documentation/realtime/language-bindings/golang/
28. gtfs-realtime-bindings/golang/gtfs/gtfs-realtime.pb.go at master - GitHub, Zugriff am März 21, 2026, https://github.com/MobilityData/gtfs-realtime-bindings/blob/master/golang/gtfs/gtfs-realtime.pb.go
29. gtfs package - github.com/OneBusAway/go-gtfs - Go Packages, Zugriff am März 21, 2026, https://pkg.go.dev/github.com/OneBusAway/go-gtfs
30. Google Cloud Run Pricing in 2025: A Comprehensive Guide - Cloudchipr, Zugriff am März 21, 2026, https://cloudchipr.com/blog/cloud-run-pricing
31. Choosing between Cloud Functions and Clod Run for a project : r/googlecloud - Reddit, Zugriff am März 21, 2026, https://www.reddit.com/r/googlecloud/comments/1jagdnm/choosing_between_cloud_functions_and_clod_run_for/
32. Cloud Scheduler pricing, Zugriff am März 21, 2026, https://cloud.google.com/scheduler/pricing
33. Cloud Run pricing | Google Cloud, Zugriff am März 21, 2026, https://cloud.google.com/run/pricing
34. Google Cloud Functions in 2025: What Teams Should Know - Cloudchipr, Zugriff am März 21, 2026, https://cloudchipr.com/blog/google-cloud-functions
35. Build a real-time analytics database with Bigtable and BigQuery - Google Cloud, Zugriff am März 21, 2026, https://cloud.google.com/solutions/real-time-analytics-for-databases
36. Dedicated MongoDB vs. BigQuery vs. Firestore : r/googlecloud - Reddit, Zugriff am März 21, 2026, https://www.reddit.com/r/googlecloud/comments/1alb2ig/dedicated_mongodb_vs_bigquery_vs_firestore/
37. Compare Google Cloud Firestore vs. Cloud SQL - G2, Zugriff am März 21, 2026, https://www.g2.com/compare/google-cloud-firestore-vs-google-cloud-sql
38. Google Cloud BigQuery vs. Google Cloud Firestore vs. Google Cloud SQL - SourceForge, Zugriff am März 21, 2026, https://sourceforge.net/software/compare/BigQuery-vs-Google-Cloud-Firestore-vs-Google-Cloud-SQL/
39. Firestore pricing - Google Cloud, Zugriff am März 21, 2026,

https://cloud.google.com/firestore/pricing
40. Work with time series data | BigQuery - Google Cloud Documentation, Zugriff am März 21, 2026, https://docs.cloud.google.com/bigquery/docs/working-with-time-series
41. What are the most cost effective Google Cloud products for running web scraping/http scripts and updating a database on a scheduled basis? : r/googlecloud - Reddit, Zugriff am März 21, 2026, https://www.reddit.com/r/googlecloud/comments/pd5d9g/what_are_the_most_cost_effective_google_cloud/
42. Running Background Tasks and Scripts as Systemd Services - DoHost, Zugriff am März 21, 2026, https://dohost.us/index.php/2025/07/30/running-background-tasks-and-scripts-as-systemd-services/
43. Creating and Managing Daemon Services in Linux using Systemd | by Shreyas Matade, Zugriff am März 21, 2026, https://medium.com/@shreyasmatade/creating-and-managing-daemon-services-in-linux-using-systemd-9ea9ac1fbf37
44. Integration of a Go service with systemd : r/golang - Reddit, Zugriff am März 21, 2026, https://www.reddit.com/r/golang/comments/84jlyb/integration_of_a_go_service_with_systemd/
45. The systemd daemon | Administration Guide | SLES 15 SP7 - SUSE Documentation, Zugriff am März 21, 2026, https://documentation.suse.com/sles/15-SP7/html/SLES-all/cha-systemd.html
46. Chapter 10. Managing Services with systemd | System Administrator's Guide | Red Hat Enterprise Linux | 7, Zugriff am März 21, 2026, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/system_administrators_guide/chap-managing_services_with_systemd
47. Backgrounding processes in Android · fyne-io fyne · Discussion #5221 - GitHub, Zugriff am März 21, 2026, https://github.com/fyne-io/fyne/discussions/5221
48. Step-by-Step Guide to Go Fyne Development | by Kanishka Meddegoda - Medium, Zugriff am März 21, 2026, https://kanishkame.medium.com/step-by-step-guide-to-go-fyne-development-321d63475a2f
49. Running a go program on Android? - Stack Overflow, Zugriff am März 21, 2026, https://stackoverflow.com/questions/28748361/running-a-go-program-on-android
50. How GoMobile Bridges Golang and Native Mobile for High-Performance Apps - RBA, Zugriff am März 21, 2026, https://www.rbaconsulting.com/blog/how-gomobile-bridges-golang-and-native-mobile-for-high-performance-apps/
51. GO! Golang and Gomobile vs Android+Kotlin and IOS+Swift. | by Igor Steblii - Medium, Zugriff am März 21, 2026, https://igorsteblii.medium.com/go-golang-and-gomobile-vs-android-kotlin-and-ios-swift-599469d7e74a
52. wilyarti/android-kotlin-golang-example-project - GitHub, Zugriff am März 21, 2026,

https://github.com/wilyarti/android-kotlin-golang-example-project
53. Go Binary on Android : r/golang - Reddit, Zugriff am März 21, 2026, https://www.reddit.com/r/golang/comments/1go3ii9/go_binary_on_android/
54. Reverse Java bindings with gomobile and fyne.io - Stack Overflow, Zugriff am März 21, 2026, https://stackoverflow.com/questions/65278073/reverse-java-bindings-with-gomobile-and-fyne-io
55. How to run a simple Go script as an Android service? - Stack Overflow, Zugriff am März 21, 2026, https://stackoverflow.com/questions/49461141/how-to-run-a-simple-go-script-as-an-android-service
56. Bound services overview | Background work - Android Developers, Zugriff am März 21, 2026, https://developer.android.com/develop/background-work/services/bound-services
57. The Influence of Public Transport Delays on Mobility on Demand Services - MDPI, Zugriff am März 21, 2026, https://www.mdpi.com/2079-9292/10/4/379